NetSim – An Interactive Graph Visualization and Algorithm Simulator

Tyler Layton

Advised by Konstantin Zuzmin

Introduction

Graph-structured data is fundamental to the world with applications ranging from **social networks** to **biological pathways.** Understanding the structural properties and visualizing these large networks is essential to our further exploration of them.

This research project presents an app which allows users to **interact** and **visualize graphs** presenting a variety of different visual and search algorithms. Such visual algorithms include a force-directed layout as well as a node contraction algorithm as a means of graph simplification.

Backend Representation

There are several common ways to represent a graph in code.

- **Adjacency matrix –** edges in a NxN Boolean grid. Simple to query but fails with large graphs.
- **Adjacency list –** storing only existing edges per node but has a slow edge look up.
- **Packed Memory Array** (PMA) **–** single block of memory which holds all edge information. Best solution for large graphs.

The backend consists of **2 lists**, one holding nodes and the other holding edges. Each node can be found based on its index/ID and holds 2 integers: an **edge index** pointing to the start of its edges in the edge list and a **degree** indicating how many edges it has. Edges are simply 1 integer which holds its destination node's ID and a label. Edges are sorted which allows for a **binary look-up (O(log deg)).** Each node is also allocated **additional space** within the edge array allowing room for new edges to be added without having to restructure the array on every edge addition.

Existing work

- **Gephi** is widely used for graph analysis offering a wide variety of visual statistics.
- **GraphViz** focuses on graphing layouts with a scripting interface but lacks a user interface.

NetSim draws inspiration from these tools targeting an interactive and accessible experience for the user. A couple of **NetSim** features include:

- **Representation table** which shows the nodes and edges which are in a graph.
- **Algorithm panel** where the user can select algorithms to run on the graph

Analysis of Packed Memory Array

Op	Adj Matrix	Adj List	PMA
Edge Insert	O(1)	O(1)	$O(\log^2(V+E))$
Edge Delete	O(1)	O(V)	$O(\log^2(V+E))$
Edge Lookup	O(1)	O(deg)	O(log deg)
Node insert	$O(V^2)$	O(1)	$O(\log^2(V+E))$
Node Delete	$O(V^2)$	O(V+E)	$O(\log^2(V+E))$
Node Lookup	O(1)	O(1)	O(1)
Memory Usage	$O(V^2)$	O(V+E)	O(V+E)

Figure 1: Complexity comparisons between Adjacency matrix, Adjacency List, and PMA

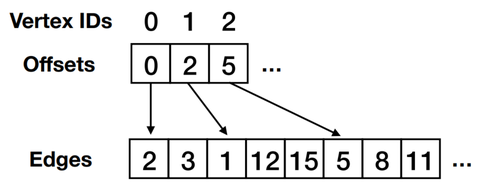

Figure 2: Visual representation of the backend using PMA

Visualization Algorithms

A large issue with graph visualization is **scalability**. Rendering thousands of nodes and edge simultaneously becomes slow and difficult to interpret. **NetSim** addresses this issue through **node contraction** which reduces rendered objects while still **preserving** the underlying graph structure.

One contraction algorithm in **NetSim** selects the **top X%** of nodes by degree contracting **Y hops** outward from these nodes shown in **Figure 3.**

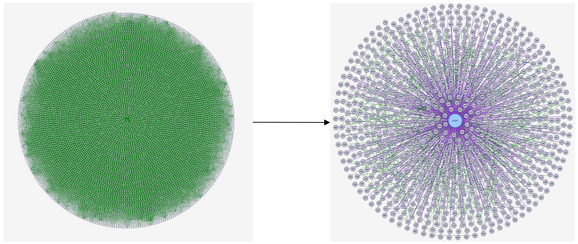

Figure 3: (Right) **– |V| = 9.5k**, **|E| = 32k** (spiral format) (Left) **– |V| = 850**, **|E| = 800** (top 30% of nodes contracted to 1 hop, spiral format)

Another challenge is determining where to **place nodes**. Simple layouts such as circular or spiral arrange nodes without any awareness of graph structure. The Scalable Force-Directed Placement **(SFDP)** algorithm solves this by simulating physical forces, connected nodes **attract** while unconnected nodes **repel**. After many iterations, the graph reaches a state of **equilibrium**, naturally **clustering** related nodes together and making the graph's structure visually apparent.

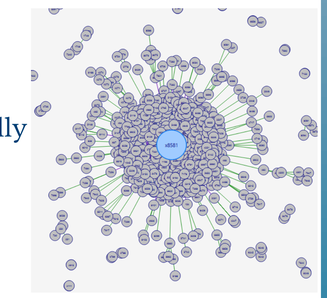

Figure 4: The contracted graph from figure 3 after running the SFDP algorithm for 100 iterations

Conclusion

NetSim shows that interactive graph visualization can be made both through **efficient** graph storage and graph **simplification**. A **packed memory array** keeps memory low with fast look-up times, while **node contraction** and **force-directed layout** make large graphs **navigable** and **easy to interpret**. Though there is room for further expansion, **NetSim** provides a solid foundation for future research into interactive graph analysis.

Future Work

NetSim only scratches the surface of what a graph visualization and algorithm simulator can offer. Many additional features can be added to enhance user experience:

- A **scripting** interface so users can write and run their own algorithms
- Ability to **save graphs** and **algorithm configurations** as a **NetSim** executable allowing work to be **resumed**
- Further performance **optimization**
- Additional visualization algorithms to help reduce graph objects
- **Visual paths** shown for the search algorithms

References

- https://itshelenxu.github.io/files/papers/pcsr.pdf
- https://networkrepository.com/bio-grid-human.php
- http://yifanhu.net/PUB/graph_draw_small.pdf
- https://juliagraphs.org/NetworkLayout.jl/stable/